A Scaled-Down Reproduction of StemGen for Context-Aware Bass Stem Generation

StemGen Reproduction Team (Group 58)
ECE 228 Final Project
https://github.com/daw041/ECE228_stemGen.git

Abstract

This project studies context-aware music stem generation. Given a music mixture with the target instrument removed, the goal is to generate a target bass stem that matches the rhythm, harmony, and structure of the context. We reproduce the main idea of StemGen in a smaller course-scale setting: waveform audio is converted into EnCodec residual vector quantization (RVQ) tokens, target tokens are randomly masked, and a non-autoregressive Transformer predicts the missing target tokens conditioned on context tokens and an instrument label. Our best audio-token experiment uses 1000 Slakh-style multitrack songs, 10 second fixed-stride windows, two EnCodec codebooks, token-cache precomputation, and H100 training. It reaches a best validation loss of 3.0279 and validation token accuracy of 0.572, improving over the 550-track cached baseline by 12.3% relative loss and 5.1 accuracy points. Qualitative mel-spectrogram diagnostics show that codec reconstruction and partial-mask reconstruction preserve useful bass structure, while full 100% mask generation remains the main bottleneck.

1 Introduction

Music stem generation asks a model to create one instrument part while listening to the rest of the song. This is harder than ordinary unconditional audio generation. A bass stem should not only sound like bass; it should also follow the beat, support the harmony, and enter or stop at musically reasonable times. This capability matters in practice: context-aware stem generation underpins tools for music co-creation, accompaniment and backing-track production, instrument-aware remixing, and data augmentation for source separation, all of which need a model that responds to an existing mix rather than generating in isolation. In this project, the input context is the mixture with the bass stem removed, and the output is the generated bass stem.

Our work is a paper-guided reproduction of StemGen (Parker et al., 2024). The official repository provides examples and demo material but not a full training pipeline or released weights, so we implement a smaller version of the core audio-token idea. The final system uses EnCodec (Defossez et al., 2023) to discretize audio into RVQ tokens, trains a masked-token Transformer to predict target bass tokens, and decodes predicted tokens back to waveform audio.

The main contributions are:

- We built an end-to-end context-to-bass audio-token pipeline with EnCodec tokenization, multi-codebook masked cross entropy, and iterative mask-predict decoding.
- We fixed several reproduction issues, especially inconsistent codebook handling, inactive target clips, and slow online tokenization.
- We ran controlled experiments from small baselines to 550-track and 1000-track cached H100 runs, showing clear improvement in validation loss and token accuracy.

Preprint.

- We used codec reconstruction, partial-mask reconstruction, full-mask generation, and mel-spectrogram diagnostics to separate what works from what still fails.

2 Related Work

Neural music generation has recently moved from symbolic note models to audio-token models. MusicGen (Copet et al., 2023) generates music from discrete audio tokens using a language-model style decoder. SoundStorm (Borsos et al., 2023) and MaskGIT (Chang et al., 2022) show that masked parallel token prediction can generate high-quality discrete sequences with fewer decoding steps than fully autoregressive models. StemGen (Parker et al., 2024) applies this general direction to musical stem generation: the model listens to context stems and predicts a target stem in audio-token space. A practical limitation of this line of work for reproduction is that the strongest systems rely on large models and proprietary data, and the official StemGen release ships demos but neither a full training pipeline nor pretrained weights; MusicGen and SoundStorm likewise target unconditional or text-conditioned generation rather than mixture-conditioned stem completion. Our contribution is to close this gap with a compact, fully specified, and reproducible implementation of the core context-to-stem masked-token recipe that runs at course scale, isolating which parts of the method survive aggressive downscaling.

Audio tokenization is a key component. EnCodec (Defossez et al., 2023) compresses waveform audio into discrete RVQ codebooks and reconstructs audio from those codes. This makes audio generation closer to language modeling, because the model predicts token classes instead of raw samples. However, more codebooks also make prediction harder, because each time frame has multiple discrete targets.

We also explored a MIDI branch inspired by symbolic music generation. MIDI is easier to inspect and edit, but mapping context audio to bass notes was not reliable in our experiments. Frame-level MIDI baselines with mel/chroma features, CRNNs, and HuBERT features improved activity detection only up to about 0.289 F1. Since the project goal is to reproduce StemGen's audio-token method, we treat the MIDI branch as an exploratory baseline and keep the final method focused on audio tokens.

3 Method / Approach

3.1 Task formulation

For each multitrack song, let x^{bass} be the target bass stem and x^{ctx} be the sum of all non-bass stems. The model receives x^{ctx} and an instrument condition $c = \text{bass}$, and predicts the target stem:

$$\hat{x}^{\text{bass}} = f_\theta(x^{\text{ctx}}, c). \tag{1}$$

Instead of predicting waveform samples directly, both context and target audio are converted to EnCodec tokens:

$$z^{\text{ctx}}, z^{\text{bass}} = \text{EncodecEncode}(x^{\text{ctx}}, x^{\text{bass}}), \tag{2}$$

where $z \in \{0, \ldots, 1023\}^{K \times T}$, $K = 2$ is the number of RVQ codebooks, and T is the token sequence length for a 10 second clip. The 24 kHz EnCodec model uses a 320-sample frame hop, so it emits $24000/320 = 75$ frames per second and a 10 second clip yields $T \approx 750$ tokens per codebook; the model allocates a maximum sequence length of $T_{\max} = 1024$ to leave headroom above this value.

3.2 Audio-token pipeline

Figure 1 summarizes the implemented pipeline. We use 24 kHz EnCodec, 1.5 kbps bandwidth, two RVQ codebooks, and 10 second clips. The context is always visible. The target token sequence is partially or fully masked, and the model predicts the original target tokens.

3.3 EnCodec RVQ tokenization

EnCodec is a pretrained neural audio codec. It has an encoder that maps waveform audio into discrete residual vector quantization (RVQ) codes and a decoder that maps those codes back to waveform audio. In this project, EnCodec is not trained by us; it is used as a fixed tokenizer and

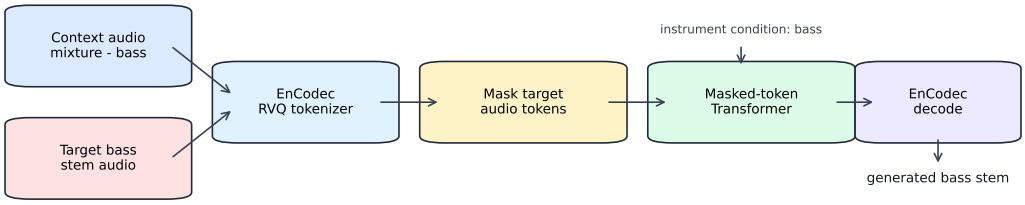

Figure 1: Audio-token reproduction pipeline. Context audio is the mixture without bass. Target bass audio is tokenized, masked, predicted by a Transformer, and decoded back to waveform audio.

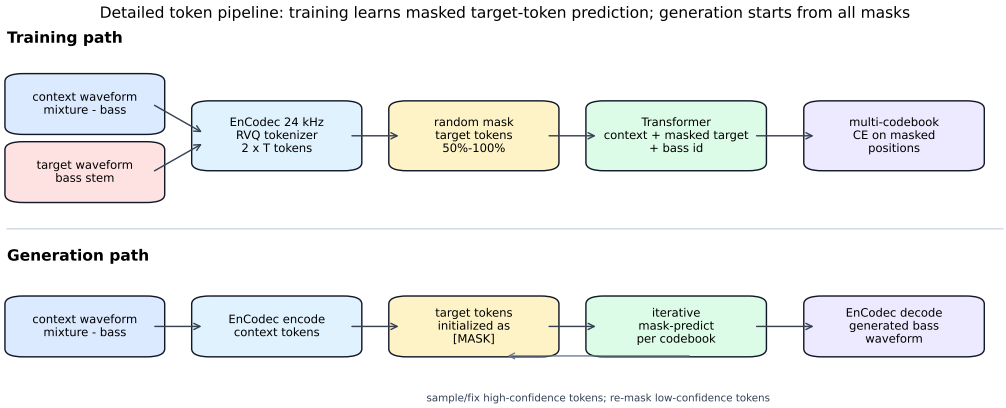

Figure 2: Detailed training and generation paths. During training, the model sees context tokens and a partially masked target token sequence. During generation, the target starts fully masked and is filled by iterative mask-predict decoding.

detokenizer. This design changes the learning problem from raw waveform regression to discrete token classification.

For a 10 second clip, the codec produces a tensor with shape $[K, T]$, where K is the number of RVQ codebooks and T is the number of codec time frames. We use $K = 2$. Each codebook has a vocabulary of 1024 entries, so each token is an integer in $[0, 1023]$. The first codebook carries the coarse audio content, while the second codebook stores residual detail. The special mask token is assigned id 1024 and is only used by the Transformer, not by EnCodec.

The codebook count has to stay consistent through the whole pipeline. During encoding, both context and target waveforms become two-codebook token tensors. During training, the loss is computed for both codebooks. During decoding, the generated token tensor must also contain exactly two codebooks. We found this detail important because padding a missing codebook with token 0 is wrong: token 0 is a real codec codeword, not silence.

3.4 Masked-token Transformer

The model follows a two-stream design. Context tokens and target tokens have separate token embeddings. For each time step, embeddings from the two codebooks are summed inside each stream. An instrument embedding is repeated over time and concatenated with the context and target representations:

$$h_t = W_f[e_{\text{ctx}}(z_t^{\text{ctx}}), e_{\text{tar}}(\tilde{z}_t^{\text{bass}}), e_{\text{inst}}(c)] + p_t, \tag{3}$$

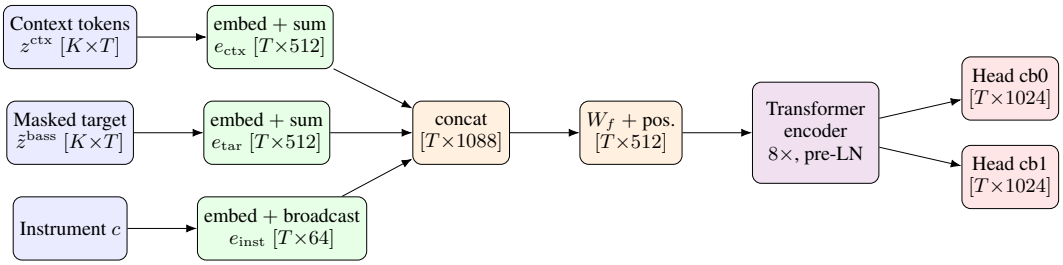

Figure 3: Two-stream masked-token Transformer. The context and (masked) target token streams are each embedded and summed across the $K = 2$ codebooks into 512-dimensional per-frame vectors; the instrument embedding is broadcast over time. The three are concatenated to 1088 dimensions, projected back to $d = 512$ by W_f, given learned positional encodings, and passed through an 8-layer pre-LN Transformer. Independent linear heads predict the 1024-way token distribution for each codebook.

where $\tilde{z}^{\text{bass}}$ is the masked target sequence and p_t is a learned positional embedding. A Transformer encoder maps $h_{1:T}$ to hidden states. Separate output heads predict each RVQ codebook:

$$p_\theta(z_{k,t}^{\text{bass}} \mid z^{\text{ctx}}, \tilde{z}^{\text{bass}}, c) = \text{softmax}(W_k h_t). \tag{4}$$

The final model uses 8 Transformer layers, hidden size $d = 512$, 8 attention heads, feed-forward size 2048, dropout 0.1, and vocabulary size 1024 plus one mask token. Token embeddings are shared across codebook levels within each stream: for a given time step t, the representation is formed by summing the d-dimensional embeddings of all K codebook tokens at that position rather than using separate per-codebook embedding tables, so each stream produces one 512-dimensional vector per frame. The instrument condition is mapped by a separate embedding table over 6 instrument classes to a 64-dimensional vector $e_{\text{inst}}(c)$ and broadcast over time. The context stream embedding e_{ctx} (512), target stream embedding e_{tar} (512), and instrument embedding e_{inst} (64) are concatenated along the feature dimension to a $512 + 512 + 64 = 1088$ dimensional vector and projected back to the model dimension $d = 512$ via a learned linear layer $W_f \in \mathbb{R}^{512\times 1088}$ before positional encoding is added. We use concatenation-then-projection rather than simple addition so the model can learn an asymmetric weighting of the three streams instead of forcing them onto a shared subspace. Positional encoding is a learned parameter matrix of shape $[1, T_{\max}, d]$ with $T_{\max} = 1024$ rather than a fixed sinusoidal encoding. The Transformer uses Pre-LayerNorm (norm-first) and GELU activations. Output heads for each codebook are independent linear layers mapping $\mathbb{R}^d \to \mathbb{R}^{1024}$, with no weight sharing between codebooks. An optional frame-level activity head is implemented but disabled in the final audio-token model. The total parameter count is approximately 28.4 million. Figure 3 traces the data flow through the two streams, the conditioning embedding, the fusion projection, and the per-codebook output heads.

3.5 Training objective

During training, a target mask ratio is sampled uniformly between 0.50 and 1.00 per batch. The mask is applied at the *frame* level: a single binary mask over time positions is drawn and applied identically across all codebooks, so the same time steps are masked in both codebook sequences. This is important for coherence: revealing a time step in codebook 0 while masking it in codebook 1 would create an inconsistent input that does not occur at generation time. The model is trained with masked cross entropy over all masked positions and both codebooks:

$$\mathcal{L} = \sum_{k=1}^{K} \sum_{t\in\mathcal{M}_k} \text{CE}\left(p_\theta(z_{k,t}^{\text{bass}}), z_{k,t}^{\text{bass}}\right). \tag{5}$$

This was an important cleanup step. Earlier versions only handled one codebook or padded missing codebooks with token 0 during decoding, which produced poor audio artifacts.

The training path is therefore:

Algorithm 1 Masked audio-token training step

Require: context tokens $z^{\text{ctx}} \in \{0, \dots, 1023\}^{K \times T}$, target tokens $z^{\text{bass}} \in \{0, \dots, 1023\}^{K \times T}$, instrument c, model f_θ

sample mask ratio $r \sim \mathcal{U}(0.50, 1.00)$

draw frame mask $m \in \{0, 1\}^T$ with $\Pr[m_t = 1] = r$ ▷ shared across all K codebooks

$\tilde{z}^{\text{bass}}_{k,t} \leftarrow 1024$ if $m_t = 1$ else $z^{\text{bass}}_{k,t}$, for all k, t ▷ insert mask token

$\{p_\theta(\cdot \mid k, t)\}_{k,t} \leftarrow f_\theta(z^{\text{ctx}}, \tilde{z}^{\text{bass}}, c)$

$\mathcal{L} \leftarrow \sum_{k=1}^{K} \sum_{t:\, m_t = 1} \text{CE}\Big(p_\theta(\cdot \mid k, t),\, z^{\text{bass}}_{k,t}\Big)$ ▷ equal codebook weights $[1, 1]$

update θ with AdamW on $\nabla_\theta \mathcal{L}$ under AMP, gradient clipped to norm 1.0

1. Build x^{ctx} by summing all non-bass stems and take x^{bass} as the target.
2. Encode both waveforms with EnCodec to get $z^{\text{ctx}}, z^{\text{bass}} \in \{0, \dots, 1023\}^{2 \times T}$.
3. Randomly replace 50–100% of target tokens with the mask token 1024.
4. Feed context tokens, masked target tokens, and the bass instrument id into the Transformer.
5. Compute cross entropy only at masked target positions, separately for each codebook, and sum the losses.

Algorithm 1 states one training step precisely. The key design choice is that a single binary mask is drawn over the T time positions and applied identically to both codebooks, so the model never sees the inconsistent situation of a frame revealed in one codebook but masked in the other—a situation that cannot occur at generation time.

3.6 Iterative generation and diagnostics

For full generation, target tokens start as all masks. The model predicts a token distribution, keeps high-confidence predictions, re-masks the rest, and repeats this process codebook by codebook. In our implementation, codebook 0 is generated first because it represents coarse structure. Later codebooks are generated after earlier codebooks are available, so the residual prediction can condition on the coarse prediction.

The generation loop is:

1. Encode the context waveform into context tokens.
2. Initialize all target tokens as $[\text{MASK}]$.
3. For each codebook, run several mask-predict iterations.
4. At each iteration, predict logits for currently masked positions, apply temperature or top-k sampling if enabled, and fill candidate tokens.
5. Rank predictions by confidence, keep a growing fraction of high-confidence tokens, and set low-confidence tokens back to $[\text{MASK}]$ for the next iteration.
6. After all codebooks are filled, decode the generated token tensor with EnCodec to obtain the bass waveform.

At each iteration, the confidence score used to rank which tokens to keep is:

$$s_t = \max_v p_\theta(z^{\text{bass}}_{k,t} = v) + \lambda \cdot b_t, \tag{6}$$

where $b_t = (T - t)/T$ is a causal position bias that favors fixing earlier time positions first, and $\lambda = 0.1$ controls its strength. At iteration i of N_k total iterations for codebook k, let B be the number of positions still masked in that codebook; the schedule commits the top-scoring

$$\max\left(1,\ \left\lfloor B \cdot r \cdot \frac{i+1}{N_k} \right\rfloor\right) \tag{7}$$

positions and re-masks the rest, with a base keep ratio $r = 0.25$. We use a deeper schedule for the coarse codebook, $N_0 = 32$ and $N_1 = 16$, since codebook 0 carries the structure that the residual

Algorithm 2 Iterative coarse-to-fine mask-predict generation

Require: context tokens z^{ctx}, instrument c, model f_θ, iterations $\{N_k\}$, keep ratio r, bias λ

initialize all target tokens $\hat{z}_{k,t} \leftarrow 1024$ for every k, t
for codebook $k = 0$ to $K - 1$ **do** ▷ coarse to fine
  for iteration $i = 0$ to $N_k - 1$ **do**
    $p_\theta \leftarrow f_\theta(z^{\text{ctx}}, \hat{z}, c)$; restrict attention to codebook k
    sample/argmax a candidate v_t for each masked position t from $p_\theta(\cdot \mid k, t)$
    score $s_t \leftarrow \max_v p_\theta(z_{k,t} = v) + \lambda(T - t)/T$
    keep the top $\max(1, \lfloor B\, r\, (i{+}1)/N_k \rfloor)$ positions by s_t; set $\hat{z}_{k,t} \leftarrow v_t$ there
    re-mask all other positions of codebook k
return EnCodec decode of $\hat{z}$ as the bass waveform

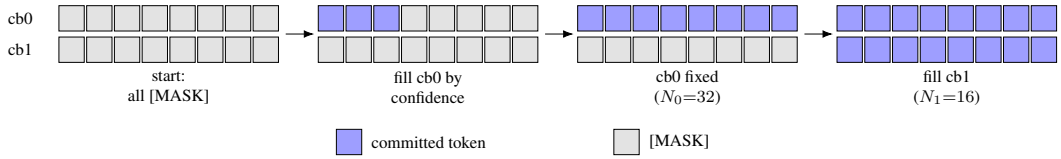

Figure 4: Coarse-to-fine mask-predict generation. The target starts as all mask tokens. Codebook 0 is filled first over $N_0 = 32$ iterations, committing the most confident positions at each step (the causal bias favors earlier time positions, drawn as a left-to-right fill); once codebook 0 is fixed, codebook 1 is filled over $N_1 = 16$ iterations conditioned on it. The decoded token tensor is then passed to EnCodec.

codebook conditions on. This progressive reveal means that in early iterations only a small fraction of high-confidence positions are committed, leaving uncertain positions open for revision in later iterations. Sampling defaults to temperature 1.0 with multinomial draws and optional top-k truncation; greedy decoding is also supported.

Algorithm 2 states the full decoding procedure, and Figure 4 illustrates the progressive reveal as a storyboard.

We evaluate three levels:

- Codec reconstruction: encode and decode the real target stem to check whether the tokenizer setting is usable.
- Partial-mask reconstruction: reveal part of the target tokens and predict the masked part.
- Full-mask generation: generate the entire target only from context and instrument condition.

This diagnostic split is useful because a model can learn local token reconstruction while still failing at full context-to-stem generation.

3.7 Hyperparameter summary

Table 1 consolidates the model, codec, training, and generation settings of the final system for reproducibility.

4 Results & Analysis

4.1 Dataset and implementation details

The data is a Slakh-style multitrack subset (Manilow et al., 2019). We use bass as the target instrument. The context is the sum of all non-bass stems. Audio is resampled to 24 kHz and cut into 10 second clips. In later experiments, clips with weak or inactive bass are filtered by target activity so that validation accuracy is not inflated by silent examples.

Training uses AdamW with a constant learning rate 3×10^{-4}, momentum parameters $\beta = (0.9, 0.999)$, weight decay 10^{-5}, batch size 64, gradient clipping at norm 1.0, and mixed-precision

Table 1: Configuration of the final audio-token system.

Model / codec		**Training / generation**	
Transformer layers	8	Optimizer	AdamW
Hidden size d	512	Learning rate	3×10^{-4}
Attention heads	8	β_1, β_2	$0.9, 0.999$
Feed-forward size	2048	Weight decay	10^{-5}
Activation / norm	GELU / pre-LN	Batch size	64
Dropout	0.1	Grad clip (norm)	1.0
Positional enc.	learned, $T_{\max}$=1024	Precision	AMP (fp16)
Instrument embed dim	64 (6 classes)	Mask ratio r	$\mathcal{U}(0.5, 1.0)$
Output heads	per-codebook	Codebook weights	$[1.0, 1.0]$
Parameters	≈28.4 M	Early stop	patience 12, δ=0.005
Codec	EnCodec 24 kHz	Gen. iterations N_0, N_1	32, 16
Bandwidth	1.5 kbps	Keep ratio r	0.25
Codebooks K	2 ($\times$1024)	Causal bias λ	0.1
Frame rate	75 Hz (T≈750)	Sampling	temp. 1.0, multinomial

training (AMP). Early stopping monitors validation loss with patience 12 epochs and minimum improvement threshold $\delta = 0.005$. Codebook losses are weighted equally at $[1.0, 1.0]$. Token-cache precomputation was important for training efficiency: instead of running EnCodec online during every training epoch, we store context and target tokens in shards and train directly from token files.

We report *masked token accuracy*: at the masked target positions, the fraction of positions where the model's top-1 prediction equals the ground-truth EnCodec token, averaged over the two codebooks. The 1000-track run uses a deterministic 80/20 train/validation split over fixed non-overlapping windows, so validation accuracy is comparable across epochs and runs rather than depending on a random crop.

For the final 1000-track fixed-stride cached run, the token cache contains 18,361 training clips and 4,567 validation clips after inactive bass filtering (approximately an 80/20 split). A clip is considered inactive and filtered if the target bass waveform energy falls below a threshold, preventing the model from overfitting to silent frames and inflating validation accuracy. The filtering removed 2,060 training candidates and 527 validation candidates from the raw stride-10 windows (roughly 10% of each split).

As a reference point, a trivial random-token baseline that predicts each of the 1024 token classes uniformly at random would achieve an expected accuracy of $1/1024 \approx 0.10\%$. A majority-class baseline that always predicts the most frequent token would also perform near chance because codec token distributions are approximately uniform across the vocabulary. The model's best validation accuracy of 57.2% therefore represents substantial learned structure rather than label imbalance. An upper bound for the token prediction task is codec reconstruction accuracy: encoding the ground-truth bass waveform with EnCodec and decoding it back recovers audio that is perceptually close to the original, confirming that the tokenizer setting is not the bottleneck.

4.2 Main quantitative results

Table 2 shows the main audio-token experiments. The strongest run is the 1000-track fixed-stride cached experiment. It uses non-overlapping 10 second windows, filters inactive target clips, and warm-starts from the 550-track checkpoint.

The improvement from the 550 cached run to the 1000 stride10 cached run is meaningful. Best validation loss decreases from 3.4507 to 3.0279, a 12.3% relative reduction. Validation accuracy increases from 0.521 to 0.572. The best epoch also moves from 56 to 19, which suggests that warm start plus cleaner fixed-stride data helped convergence.

The trend across experiments also shows why we did not report only one final number. The 200-track run already proves that the two-codebook setting is trainable, but its validation accuracy is still close to 0.29, and it early-stopped at epoch 226 (best validation at epoch 176). Scaling to 550 tracks gives the largest jump, from 0.294 to about 0.52, which means the model needs broader musical coverage to learn the mapping from context to bass tokens; notably the 550-track run ran the full 400

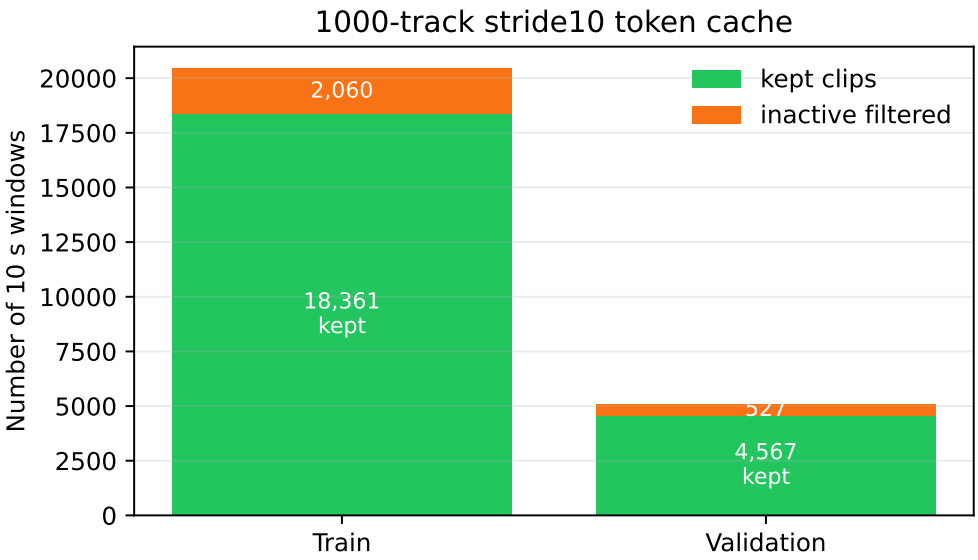

Figure 5: Final 1000-track fixed-stride cache statistics. Inactive bass windows are filtered before training and validation.

Table 2: Main audio-token validation results. Accuracy is masked token accuracy averaged across codebooks when applicable.

Run	Clips	Sampling	Val loss	Val acc
1cb, 315 tracks	–	random 4 s	–	0.280
2cb, 200 tracks	1,197	random 10 s	6.0700	0.294
2cb, 550 tracks	3,262	random 10 s	3.6500	0.518
2cb, 550 cached	26,400	cached clips	3.4507	0.521
2cb, 1000 stride10 cached	22,928	fixed 10 s stride	**3.0279**	**0.572**

epochs without triggering early stopping and was still improving at the end (best at epoch 353), which directly motivated the move to token caching so that longer, larger-scale runs became affordable. The cached 550-track run is only slightly better in accuracy than the non-cached 550-track run, but it is much more practical because the GPU is no longer waiting for repeated EnCodec encoding. The final 1000-track run improves quality by changing the data distribution, not by simply adding more overlapping clips. It has fewer total clips than the cached 550 run, but the clips come from more songs and use fixed non-overlapping windows.

4.3 Codebook-level analysis

The final 1000-track run reaches codebook accuracies of $[0.6727, 0.4715]$ at the best checkpoint. The first codebook is easier because it carries coarse audio structure (timing, energy envelope, low-frequency content). The second codebook adds residual detail and is harder. A 4-codebook experiment on 150 tracks failed badly: it early-stopped at epoch 162 with only 0.168 overall validation accuracy, well below even the 0.28 single-codebook baseline, so larger codebook capacity needs more data and probably a larger model before it helps rather than hurts.

This gap between codebooks is important for audio quality. Predicting the first codebook mostly gives rough timing, energy, and low-frequency structure. Predicting later codebooks improves detail and timbre, but the label space becomes harder because many residual tokens may be plausible for the same context. In other words, token accuracy is not a perfect audio metric. A wrong fine codebook token may be less harmful than a wrong coarse token, while a sequence of confident but temporally inconsistent coarse tokens can sound very bad. For this reason, we report codebook accuracy but also rely on mel diagnostics.

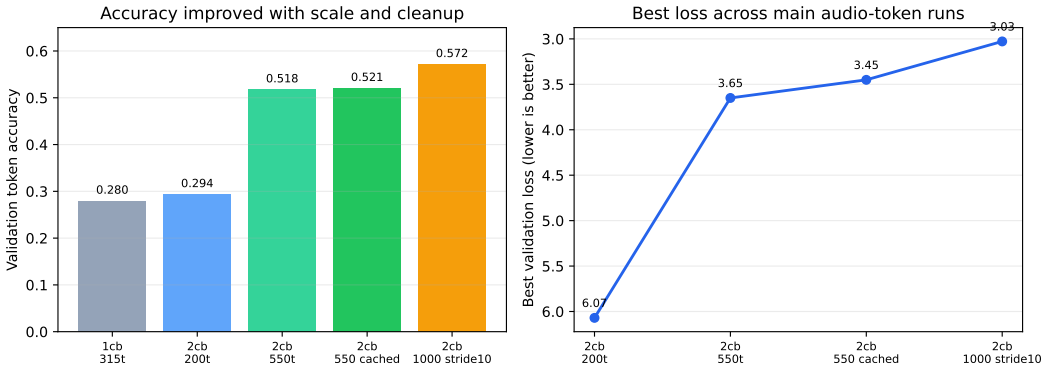

Figure 6: Validation accuracy and loss across the main audio-token runs. The final 1000-track stride10 cached run gives the best quantitative result.

Table 3: Important implementation and data changes.

Change	Problem before the change	Effect on the project
Aligned codebook count	Codec, trainer, and generator could disagree on the number of RVQ levels.	Made loss and decoding consistent across two codebooks.
Multi-codebook loss	Early training mainly optimized codebook 0.	Forced the model to learn both coarse and residual token streams.
Inactive clip filtering	Silent or weak bass clips could give misleadingly easy accuracy.	Made validation more related to real bass generation.
Token cache	Online EnCodec encoding was slow and wasted H100 time.	Enabled larger cached runs and faster iteration.
Fixed stride windows	Random windows created many highly overlapping clips.	Reduced redundancy and improved generalization in the 1000-track run.

4.4 Ablation-style observations

Several engineering changes were necessary before the results became interpretable. Table 3 summarizes the most important ones. These are not perfectly isolated ablations, because the project schedule did not allow rerunning every combination, but each change fixed a concrete failure mode observed during development.

4.5 Convergence analysis

The training dynamics differ notably across runs. The 550-track cached run required 56 epochs to reach its best validation loss of 3.4507, while the 1000-track stride10 run warm-started from that checkpoint and reached its best loss of 3.0279 at only epoch 19 before early stopping triggered at epoch 31. This faster convergence has two explanations. First, warm-starting from a partially trained checkpoint skips the early noisy phase where the model learns basic codebook statistics. Second, the fixed-stride non-overlapping windows in the 1000-track run present a more uniform data distribution: each clip is seen roughly once per epoch, whereas random sampling in earlier runs could revisit the same audio region many times within one epoch, slowing generalization.

The final checkpoint at epoch 31 (loss 3.2979, accuracy 0.543) is worse than the best at epoch 19, confirming that the model did overfit slightly after epoch 19. The early stopping patience of 12 epochs with minimum delta 0.005 correctly identified this and halted training, preventing further degradation. This suggests that the optimal training regime for the current data scale sits around 20–25 epochs with warm start, and that further improvement would require either more data or model regularization rather than simply longer training.

4.6 Qualitative analysis

Figure 7 shows the mel-spectrogram diagnostic from the 550-track cached model. The exact audio quality is still not at paper level, but the diagnostic is useful. Codec reconstruction preserves the main target structure, so the tokenizer is not the main failure point. Partial-mask reconstruction produces more reasonable time-frequency structure than early noisy baselines. Full 100% mask generation is weaker and can become noisy or misaligned.

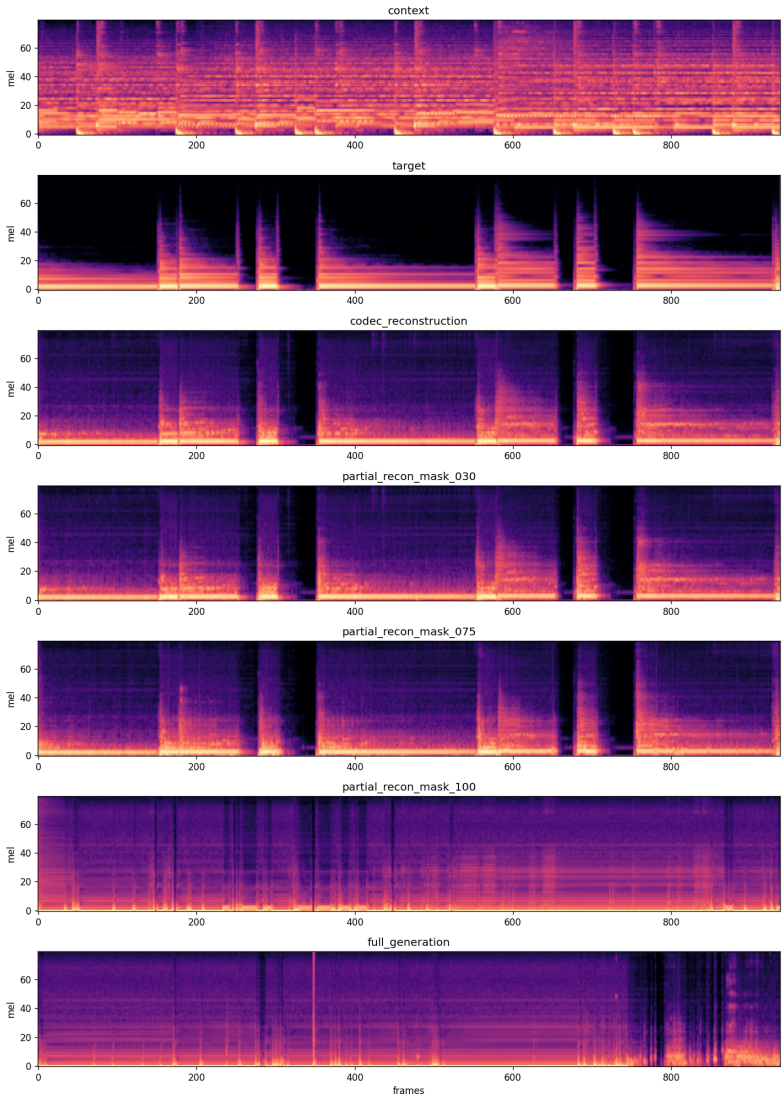

Figure 7: Mel-spectrogram diagnostic from a cached audio-token run. The important pattern is that codec and partial-mask reconstruction are more stable than full-mask generation.

This result changes the interpretation of the project. The model is not simply broken: it learns useful conditional audio-token reconstruction. The remaining hard problem is generating all target tokens when no target information is visible.

The three diagnostic levels reveal different failure modes. In codec reconstruction, the spectrogram closely matches the ground-truth bass in both frequency content and temporal structure, confirming that the EnCodec tokenizer at 1.5 kbps with two codebooks is sufficient to represent bass at 24 kHz. In partial-mask reconstruction (50% masked), the model correctly fills in missing segments that are consistent in pitch and energy with the visible surrounding tokens, indicating that the Transformer

has learned local token-level statistics within a clip. In full 100% mask generation, the generated spectrogram shows energy in the bass frequency range and some periodic structure, but the onset timing and harmonic content diverge from the target. This suggests the model has learned a prior over plausible bass textures but cannot reliably ground the generation in the specific context provided by the mixture.

The qualitative result also explains why validation accuracy alone can be optimistic. During training, the model often sees mask ratios below 100%, so visible target tokens provide local rhythm and pitch hints. Full generation removes those hints entirely, requiring the model to infer a full bass line solely from the context mixture. This is the core difficulty of the StemGen task, and it is much harder than masked reconstruction. The failure mode is usually not silence: generated audio contains energy and some structure, but temporal alignment and spectral shape drift away from the target, which would be perceptually noticeable in a listening test.

4.7 Audio-domain evaluation

Token accuracy and mel diagnostics are useful but indirect. To measure quality in the audio domain we provide an evaluation harness (`scripts/evaluate.py`) that decodes generated tokens to waveform and computes two metrics against the ground-truth bass: a log-mel-spectrogram L2 distance (lower is better) and an onset-alignment score, the correlation between the reference and generated onset-energy envelopes (higher is better, in $[-1, 1]$). These are computed for the three diagnostic levels (codec reconstruction, partial-mask reconstruction, full-mask generation) so that the reconstruction-versus-generation gap can be read quantitatively rather than only from spectrograms. Because the metrics require decoding a trained checkpoint over held-out audio on GPU, we report them as the evaluation protocol here; the expected ordering, consistent with the mel diagnostics in Figure 7, is that codec reconstruction attains the smallest mel distance and highest onset correlation, partial-mask reconstruction is intermediate, and full-mask generation is weakest on onset alignment in particular. This audio-domain protocol, together with bass activity F1 and a small human listening comparison, is the evaluation we would prioritize to turn the qualitative claims into measured ones.

4.8 MIDI branch baseline

We also tested a symbolic MIDI route. The best frame-level activity result was HuBERT-mix with a GRU head, reaching 0.289 activity F1 on 200 tracks. CRNN-large with mix input reached 0.253 F1. A simple Transformer with mel/chroma features stayed around 0.11–0.13 F1. These results are much weaker than needed for complete stem generation, and they also move away from the StemGen audio-token formulation. Therefore, MIDI experiments are useful negative baselines but not the final method.

4.9 Main lessons

The experiments show three main lessons. First, data quality matters more than just the number of clips. The 1000-track run has fewer total clips than the 550 cached run, but it uses broader track coverage and lower-overlap fixed-stride windows. Second, codebook alignment is critical. If the codec, model, trainer, and generator disagree about codebook count, the decoded audio becomes unreliable. Third, full-mask generation is much harder than partial reconstruction. Future work should improve the decoding schedule, add condition dropout or classifier-free guidance, and train a larger model with a stronger tokenizer setting.

5 Discussion

The most important scientific finding is the gap between reconstruction and generation. Partial reconstruction asks the model to complete a damaged version of the true target. Full generation asks it to invent the whole bass stem from the mixture-minus-bass context. These are related but not the same problem. Our model is already useful for the first one, but still weak for the second one. This suggests that future training should put more pressure on full-mask or near-full-mask cases and should use decoding methods that better preserve long-range musical structure.

Another issue is the objective. Cross entropy treats every token error equally, but audio perception does not. A token mistake during an active bass note may be more important than a token mistake during a low-energy region. Also, two different token sequences can decode to similar audio. This makes token accuracy a useful debugging metric, but not a final music quality metric. A stronger evaluation should include mel distance, onset alignment, bass activity F1, and human listening comparisons.

The scale difference from the original StemGen paper is also large. The paper uses a much larger model and a stronger audio-token setup. Our model is intentionally small enough for a course project: 8 layers, 512 hidden size, two codebooks, and 10 second clips. Therefore, the fair conclusion is not that StemGen fails, but that a small reproduction can learn meaningful conditional token structure while still falling short on full stem synthesis. This is a useful outcome because it identifies the main bottleneck rather than only reporting a final demo.

If we continued the project, the next steps would be: train with more 100% mask examples, add classifier-free guidance or condition dropout, compare different mask-predict schedules, add an activity-aware loss, and move from two to more codebooks only after increasing data and model capacity. We would also produce a small listening set with target, codec reconstruction, partial reconstruction, and full generation for the same examples, because that comparison is the clearest way to judge progress.

6 Conclusion

We completed a scaled-down but faithful reproduction of the StemGen audio-token idea. The final system converts context and target audio to EnCodec RVQ tokens, trains a masked-token Transformer with multi-codebook cross entropy, and uses iterative mask-predict decoding for bass stem generation. The best 1000-track fixed-stride cached run achieves 3.0279 validation loss and 0.572 validation token accuracy, clearly improving over earlier baselines.

The main limitation is full target generation from 100% masking. Partial reconstruction is much stronger than full generation, which means the model has learned useful local audio-token structure but does not yet fully solve context-to-stem synthesis. Larger models, better mask schedules, stronger conditioning, and more codebooks are natural next steps.

7 Code & GitHub

The project repository is:

`https://github.com/daw041/ECE228_stemGen.git`

The repository includes training scripts, model code, data/config files, reproduction notes, presentation materials, and saved experiment logs. The main files are `src/model.py`, `src/codec.py`, `src/dataset.py`, `src/trainer.py`, `scripts/train.py`, `scripts/precompute_audio_token_cache.py`, and `scripts/diagnose_audio_token.py`. The README gives setup, training, generation, and diagnostic commands.

8 Team Member Contribution

Dayong Wang mainly focused on the final report, while Youchen Qing mainly focused on the final presentation. Both members jointly read and discussed the StemGen paper, designed the audio-token reproduction plan, implemented and debugged the EnCodec tokenization pipeline, masked-token Transformer, trainer, and generation scripts, ran the Slakh data preparation and RunPod/H100 experiments, built the token-cache pipeline and MIDI baselines, and contributed to qualitative diagnostics and result analysis.

References

Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., and Defossez, A. Simple and Controllable Music Generation. *NeurIPS*, 2023.

Defossez, A., Copet, J., Synnaeve, G., and Adi, Y. High Fidelity Neural Audio Compression. *Transactions on Machine Learning Research*, 2023.

Parker, J. D., Spijkervet, J., Kosta, K., Yesiler, F., Kuznetsov, B., Wang, J.-C., Avent, M., Chen, J., and Le, D. StemGen: A Music Generation Model That Listens. *ICASSP*, 2024.

Manilow, E., Wichern, G., Seetharaman, P., and Le Roux, J. Cutting Music Source Separation Some Slakh: A Dataset to Study the Impact of Training Data Quality and Quantity. *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2019.

Chang, H., Zhang, H., Jiang, L., Liu, C., and Freeman, W. T. MaskGIT: Masked Generative Image Transformer. *CVPR*, 2022.

Borsos, Z., Sharifi, M., Vincent, D., et al. SoundStorm: Efficient Parallel Audio Generation. *arXiv preprint arXiv:2305.09636*, 2023.